- **Server:** creates or loads global parameters and selects the clients for the round.
- **Distribution:** sends the current global parameters and training configuration to selected clients.
- **Local optimization:** each client trains on its own training split for a defined number of local epochs.
- **Upload:** clients return model parameters or parameter deltas plus the sample count and required metadata.
- **Aggregation:** the server combines the eligible client updates to form the next global model.
- **Evaluation:** validate the global model, record metrics, and continue until the stopping rule is met.

Terminology

Term	Meaning
Client	A participating data owner; here, a simulated hospital.
Global model	The shared model maintained by the federated server.
Local model	A client's copy of the model during local training.
Communication round	One cycle of distribution, local training, upload, and aggregation.
Local epoch	One pass through the client's local training data.
Client drift	Divergence of local model updates, often more pronounced with heterogeneous data.

Raw MRI images should not be sent to the aggregation server in this workflow. Define exactly what is transmitted, logged, and retained.

Algorithm 1: FedAvg

Federated Averaging — the baseline algorithm.

Definition: FedAvg is a federated optimization algorithm in which clients perform local training and the server combines their resulting model parameters using a sample-size-weighted average. It was introduced by McMahan and colleagues in 2017.

Server aggregation equation

$$w_{t+1} = \Sigma_{k \in S_t} (n_k / n_S) \cdot w_{t+1}^{k}$$

Here, w_{t+1} is the new global model; S_t is the set of participating clients; n_k is the number of training examples used by client k; n_S is the total number of examples across participating clients; and w_{t+1}^{k} is client k's locally trained model.

FedAvg in your brain tumor project

- Initialize one CNN (or selected image model) on the server.
- Copy the global weights to each selected hospital client.
- Train each local copy using that hospital's MRI training images and labels.
- Send local weights and the number of training examples to the server.
- Compute the weighted average of corresponding model parameters.
- Use the aggregated weights as the next global model and evaluate it.

Strengths

- Simple and widely used baseline.
- Sample-size weighting accounts for different client dataset sizes.
- Provides a reference point for comparing heterogeneity-aware methods.

Limitations

- When hospital class distributions differ, local updates can point in different directions.
- A client with many samples can have greater influence in weighted aggregation.
- Performance and convergence may degrade under strong non-IID data.

Implementation caution: average model parameters, not client accuracy scores or confusion matrices. Ensure all clients use the same model architecture and compatible parameter ordering.

Algorithm 2: FedProx

Federated Proximal — a method designed to address client heterogeneity.

Definition: FedProx extends the local optimization objective with a proximal penalty that discourages a client's local model from moving too far from the global model received at the start of the round. It was introduced by Li and colleagues in 2020.

Local objective

$$\min_w F_k(w) + (\mu / 2) \cdot ||w - w_t||^2$$

$F_k(w)$ is the client's local training loss; w is the local parameter vector being optimized; w_t is the global parameter vector at the beginning of the round; and μ (mu) controls the strength of the proximal penalty.

How it works in your project

- The server sends the current global model to each hospital client.
- Each client trains on its local MRI data while adding the proximal penalty to its normal classification loss.
- The penalty discourages excessive divergence from the starting global weights.
- Clients send their trained weights to the server.
- The server typically uses the same sample-size-weighted aggregation as FedAvg.
- The process repeats, and the global model is evaluated.

Understanding μ

μ setting	Interpretation
0	No proximal penalty; local objective reduces to ordinary local training.
Small positive value	Gentle penalty; allows more local adaptation.
Larger positive value	Stronger constraint on local movement; may help or may over-constrain learning.

There is no universally best μ. Tune it using validation data, document the search range, and avoid selecting it using the final test set.

Potential benefit and limitation

FedProx can improve stability when clients have different data distributions or system capabilities, but it does not guarantee higher accuracy than FedAvg. A large μ can restrict useful local adaptation; a small μ may have little effect.

Other federated-learning algorithms

Which methods are relevant, and what should be included in this project?

Algorithm	Core idea	Relevance to this project
FedAvg	Weighted average of locally trained model parameters.	Essential baseline; implement first.
FedProx	Adds a proximal penalty to local training.	Primary comparison for non-IID hospital data.
SCAFFOLD	Uses control variates to correct client drift.	Useful advanced comparison if time and implementation capacity allow.
FedNova	Normalizes client updates to address differences in local work.	Relevant if clients perform different amounts of local training.
FedOpt (server optimizer)	Uses server-side optimization such as adaptive methods to update global weights.	Optional advanced experiment after a reliable baseline.
Personalized FL	Allows a shared representation with client-specific adaptation.	Relevant if hospital-specific performance is a major objective.

Recommended scope

For a clear and defensible project, implement **FedAvg and FedProx first**. They create a manageable comparison and directly connect to your planned IID and Dirichlet label-skew scenarios. Add SCAFFOLD or personalization only after the core pipeline is correct and reproducible.

Do not compare algorithms with different model architectures, data leakage, inconsistent preprocessing, or different test sets. Keep the experimental conditions matched so the algorithm is the main changing factor.

Algorithm selection should follow the research question, not the number of algorithms implemented.

IID and non-IID data: Dirichlet label skew

Why α = 0.1, 0.5, and 1.0 matter in a multi-hospital simulation.

IID means clients have approximately similar data distributions. **Non-IID** means clients' data distributions differ. In this project, label skew means the proportion of tumor classes varies across hospital clients.

Dirichlet partitioning uses a concentration parameter α to sample class proportions for each client. For the common symmetric Dirichlet construction, a smaller α tends to produce more uneven class proportions; a larger α tends to produce more even proportions. This is a tendency, not an exact guarantee for each run.

Scenario	Expected tendency	Research purpose
IID	Similar class proportions across clients.	Reference condition; easier distribution setting.
Dirichlet α = 0.1	Strong class imbalance across clients.	Stress-test severe label heterogeneity.
Dirichlet α = 0.5	Moderate-to-strong heterogeneity.	Test intermediate difficulty.
Dirichlet α = 1.0	Less concentrated proportions than smaller α values.	Test a milder non-IID setting.
Natural-source	Preserves each source hospital's observed distribution.	Compare synthetic partitions with source-wise partitions.

Important interpretation

Alpha controls the class-proportion distribution, not the total number of images assigned to each client. Your implementation should allocate every sample exactly once within each pooled-training scenario. Check class counts and client sample totals after partitioning.

The natural-source scenario is different from synthetic IID/Dirichlet partitions: it preserves source-level groupings. If your clients are created from one public dataset, describe them as simulated hospitals, not actual hospitals.

Suggested experiment grid

Data setting	FedAvg	FedProx
IID	Run	Run
α = 0.1	Run	Run
α = 0.5	Run	Run
α = 1.0	Run	Run
Natural source	Optional / recommended	Optional / recommended

How to implement this in your notebook

Map the concepts to a practical end-to-end workflow.

Stage	Implementation task	Checks before moving on
M0 — Setup	Set random seeds, install/import libraries, define paths and configuration.	Environment and versions recorded.
M1 — Data	Load images/labels, inspect classes, create train/validation/test splits, then create client partitions.	No duplicate leakage across splits; labels correct; client counts recorded.
M1.7 — Scenarios	Create natural-source, IID, and Dirichlet partitions.	All expected samples accounted for; DataFrame/tensor types valid.
M2 — Model	Define one shared CNN/image model and loss function.	Same architecture and preprocessing for all clients.
M3 — Local training	Implement client training for FedAvg and FedProx.	Only local training data used; FedProx includes μ penalty.
M4 — Server	Distribute weights and aggregate client updates.	Correct sample-size weights; state-dict keys and tensor shapes match.
M5 — Evaluation	Evaluate global model on held-out test data and report metrics.	Test set never used for training or hyperparameter selection.
M6 — Experiments	Run matched algorithm × data-setting × seed combinations.	Record configuration, results, runtime, and failures.

Practical implementation notes

- Use a single, fixed class-to-index mapping across all clients.
- Apply identical image resizing, normalization, and augmentation policy. Random augmentation should be applied only to training data.
- Keep a validation split for model selection and a final untouched test split for reporting.
- For each round, log client sample counts, local loss, global metrics, and aggregation weights.
- For FedProx, calculate the proximal term against a frozen copy of the global parameters at the start of that round.
- Save checkpoints and experiment configurations so runs can be reproduced.

Data leakage warning

Do not randomly split images into train/test after creating clients if near-duplicate images or patient-level relationships can cross partitions. If patient identifiers are available, split by patient before creating client partitions. If identifiers are unavailable, state this limitation.

Evaluation metrics and result reporting

Measure classification quality, class-wise behavior, and training behavior.

Metric	What it measures	Why it matters for brain MRI classification
Accuracy	Fraction of all predictions that are correct.	Useful overall summary, but can hide minority-class errors.
Precision	Of predictions for a class, how many are correct.	Helps understand false-positive predictions for each class.
Recall / sensitivity	Of true examples in a class, how many are detected.	Important to understand missed cases in each class.
F1-score	Harmonic mean of precision and recall.	Balances precision and recall; report macro-F1 for class balance.
Confusion matrix	Counts correct and incorrect class predictions.	Shows which tumor classes are confused.
ROC-AUC	Ranking/discrimination measure, usually one-vs-rest for multiclass.	Can complement threshold-based metrics; explain averaging method.
Convergence	Change in loss/metrics over communication rounds.	Shows learning stability and training progress.

Recommended reporting

- Report macro precision, macro recall, macro-F1, accuracy, and a confusion matrix.
- For multiclass AUC, state whether it is one-vs-rest and macro- or weighted-averaged.
- Plot global validation metrics by communication round.
- Report final test results only after selecting the model and settings using validation data.
- Repeat runs with multiple random seeds; report mean and standard deviation.
- Include per-client metrics where possible so strong global results do not conceal poor performance at one client.

A note on medical interpretation

A high benchmark score does not establish clinical safety or diagnostic usefulness. The project is an educational/research classification experiment. Clinical use would require independent external validation, representative multi-site data, expert oversight, regulatory and ethics review, and prospective evaluation.

Privacy, security, and responsible research

Federated training reduces raw-data movement but does not remove all risks.

Federated learning is often described as privacy-preserving because raw images remain at the client. However, model updates may reveal information about local training examples under some threat models. The project should describe the privacy properties accurately and avoid claiming that FL alone guarantees privacy.

Risk / issue	Practical mitigation or reporting action
Update leakage	Consider secure aggregation and/or differential privacy if within scope; document what is actually implemented.
Unauthorized access	Use authenticated access, least-privilege permissions, and secure storage.
Transport exposure	Use encrypted communication in a real deployment.
Data misuse	Use properly licensed datasets, approvals, and data-use conditions.
Class imbalance	Report per-class counts, macro metrics, and minority-class performance.
Dataset bias	Discuss source, demographics if known, scanner/protocol variation, and representativeness limits.
Overclaiming	Call clients simulated hospitals if partitions are synthetic; do not imply clinical validation.

Reproducibility checklist

- Record dataset name/version, license, class labels, image counts, and preprocessing.
- Record client partition method, alpha, seed, and per-client class counts.
- Record model architecture, optimizer, learning rate, batch size, local epochs, rounds, and μ.
- Record aggregation rule, client participation, and checkpoint selection rule.
- Keep training, validation, and test roles separate.
- State limitations, including simulated clients and absence of clinical deployment if applicable.

Recommended experiment plan and research narrative

A manageable plan for a clear, reproducible project report.

Core comparison matrix

Data partition	FedAvg	FedProx	Purpose
IID	✓	✓	Baseline with similar client distributions.
Dirichlet $\alpha = 0.1$	✓	✓	Severe label-skew stress test.
Dirichlet $\alpha = 0.5$	✓	✓	Intermediate heterogeneity.
Dirichlet $\alpha = 1.0$	✓	✓	Milder non-IID comparison.
Natural source	Optional	Optional	Preserve source-level client distributions.

Suggested research questions

- How does increasing hospital label heterogeneity affect global classification performance?
- Does FedProx improve convergence or class-wise performance compared with FedAvg under strong label skew?
- Do global metrics hide differences in performance across individual clients?
- How sensitive is FedProx to the proximal coefficient μ?

Suggested project title

“A Comparative Study of Federated Averaging and Federated Proximal Optimization for Brain Tumor MRI Classification under Non-IID Data Distributions.”

Suggested abstract structure

1) Background and motivation; 2) objective; 3) dataset and client simulation; 4) model and preprocessing; 5) FL algorithms (FedAvg/FedProx); 6) IID and Dirichlet partition settings; 7) evaluation metrics; 8) measured results; 9) limitations and conclusion.

Do not write a result claim such as “FedProx outperforms FedAvg” until your experiments support it. A valid paper may find that the outcome depends on alpha, μ, class balance, or model configuration.

Glossary and references

Core terminology and foundational papers to cite.

Term	Definition
Federated learning	Collaborative model training across clients without centralizing their raw local training data.
Client	A participating institution or data owner; represented by a simulated hospital in this project.
Non-IID	Client datasets do not follow identical distributions.
Label skew	Clients have different proportions of target classes.
Dirichlet α	Concentration parameter controlling variability of class proportions across clients.
FedAvg	Local training followed by sample-size-weighted model averaging.
FedProx	Local optimization with a proximal penalty to limit deviation from global parameters.
Communication round	One server-to-client distribution, local-training, and aggregation cycle.
Macro-F1	Average of class-wise F1 scores, giving each class equal weight.

Foundational references

1. McMahan, B. et al. (2017). *Communication-Efficient Learning of Deep Networks from Decentralized Data.* Proceedings of AISTATS. Introduces the FedAvg approach.

2. Li, T. et al. (2020). *Federated Optimization in Heterogeneous Networks.* Proceedings of Machine Learning and Systems (MLSys). Introduces FedProx.

3. Kairouz, P. et al. (2021). *Advances and Open Problems in Federated Learning.* Foundations and Trends in Machine Learning. Broad survey of FL methods, privacy, and open challenges.

4. Karimireddy, S. P. et al. (2020). *SCAFFOLD: Stochastic Controlled Averaging for Federated Learning.* Proceedings of ICML. Introduces a method to correct client drift.

Final takeaway

For this brain tumor project, build a reliable image-classification pipeline first, then evaluate FedAvg and FedProx under matched IID and non-IID client partitions. Report class-wise and per-client performance, preserve a held-out test set, and describe privacy and clinical limitations accurately.

End of guide